

BAB II

METODOLOGI PENELITIAN

2.1 Bahan dan Alat Penelitian

Dalam pelaksanaan penelitian dan pengembangan arsitektur *Time-Aware Hybrid Ranking System* ini, penulis menggunakan sejumlah bahan berupa spesifikasi data serta perangkat keras dan perangkat lunak sebagai alat penunjang operasional penelitian.

2.1.1 Bahan Penelitian

Objek utama yang digunakan dalam penelitian ini adalah *dataset* yang merepresentasikan entitas, relasi, dan rekam jejak interaksi (*engagement*) pengguna pada sebuah platform konten. Dikarenakan penelitian ini berpusat pada optimasi algoritma di sisi *backend*, basis data yang digunakan mencakup data operasional maupun data sintetis (*synthetic data*) yang dirancang khusus untuk menyimulasikan beban kerja lingkungan produksi (*production environment*). Klasifikasi spesifikasi data tersebut diuraikan sebagai berikut:

Tabel 2.1.1 Bahan penelitian

No	Tabel Basis Data	Estimasi Volume
1	Konten	5000 baris
2	Pengguna	100 akun
3	Kategori Konten	5 kategori
4	Relasi Ko	5, 500 baris
5	Skor Minat Pengguna	Dinamis

No	Tabel Basis Data	Estimasi Volume
6	Preferensi Eksplisit	dinamis
7	Riwayat Interaksi	dinamis

1. Data Metadata Konten

Merupakan atribut fundamental yang mendefinisikan identitas suatu item konten. Entitas ini mencakup identifikasi unik, atribut deskriptif, serta parameter temporal berupa *timestamp* publikasi. Parameter temporal ini diposisikan sebagai variabel penentu yang krusial dalam mengkalkulasi penalti usia konten melalui fungsi peluruhan eksponensial (*Exponential Time Decay*).

2. Data Kategori dan Vektor Preferensi Pengguna

Merupakan data relasional yang memetakan afiliasi suatu konten terhadap entitas kategori tertentu, sekaligus merepresentasikan matriks ketertarikan historis pengguna terhadap kategori tersebut. Data ini difungsikan sebagai parameter masukan komputasi (*computational input*) untuk membentuk representasi ruang vektor dalam kalkulasi algoritma *Cosine Similarity*.

3. Data Metrik Interaksi (*Engagement Metrics*)

Merupakan himpunan variabel kuantitatif yang merekam umpan balik perilaku pengguna (*user behavioral feedback*), baik secara eksplisit maupun implisit. Variabel turunan ini diekstraksi dari akumulasi metrik seperti frekuensi tayangan (*Watch Count*), tingkat adopsi templat (*Used/Recreate Count*), presensi preferensi (*Like Count*), serta nilai rata-rata ulasan (*Rating Avg*). Seluruh atribut

kuantitatif ini selanjutnya akan dinormalisasi guna merumuskan rasio kualitas interaksi konten secara proporsional.

2.1.2 Alat Penelitian

Instrumen penelitian yang digunakan mencakup seluruh infrastruktur komputasi, baik pada lapis perangkat keras (*hardware*) maupun perangkat lunak (*software*). Infrastruktur ini difungsikan untuk memfasilitasi tahapan perancangan arsitektur, implementasi kode sumber (*source code*), hingga simulasi pengujian kinerja algoritma perankingan pada lingkungan pengembangan lokal (*local development environment*).

1 Perangkat Keras (*Hardware*)

Perangkat keras diposisikan harus mampu menopang beban komputasi konkuren, dan pemrosesan basis data relasional. Untuk mendukung latensi rendah dan throughput tinggi pada lingkungan produksi nyata, arsitektur backend ini ke depannya dikembangkan menggunakan mekanisme load balancing. Hal ini merujuk pada penelitian Subagio, Hatta, dan Asih (2021) yang membuktikan bahwa metode tersebut sangat esensial untuk mendistribusikan beban web server secara terstruktur dan real-time [20]. Spesifikasi instrumen fisik yang dialokasikan dalam penelitian ini adalah:

Tabel 2.1.2. Perangkat Keras

Komponen	Spesifikasi
Processor	Macintosh berbasis arsitektur x86_64 (Prosesor Intel)
RAM	16GB DDR4

Komponen	Spesifikasi
RAM	16GB DDR4
Sistem Operasi	MacOS

Konfigurasi ini dipilih untuk menjamin kelancaran alokasi sumber daya saat menjalankan eksekusi konkurensi bahasa Go (*goroutines*), simulasi *server* basis data, serta *cache in-memory* secara bersamaan tanpa memicu memory bottleneck. Penggunaan SSD juga diperlukan untuk meminimalkan latensi I/O pada fase Candidate Generation yang melibatkan kueri agregasi.

2 Perangkat Lunak (*Software*)

Perangkat lunak merupakan instrumen logikal yang digunakan sepanjang siklus pengembangan perangkat lunak (*Software Development Life Cycle*). Lingkungan komputasi dan tumpukan teknologi (*tech stack*) yang digunakan diuraikan sebagai berikut:

a). Sistem Operasi dan Lapisan Virtualisasi

Sistem operasi macOS digunakan sebagai lingkungan komputasi utama (*primary environment*). Sistem operasi ini memfasilitasi seluruh tahapan pengembangan secara *native*, mulai dari penulisan kode sumber, kompilasi program, eksekusi layanan basis data, hingga simulasi *server* lokal untuk pengujian

b). Bahasa pemrograman Go (Golang)

Bahasa ini dipilih secara spesifik karena karakteristiknya yang dikompilasi secara statis (*statically compiled*) serta memiliki manajemen konkurensi yang tangguh, menjadikannya sangat ideal untuk memproses algoritma perankingan

hibrida yang menuntut latensi rendah (*low-latency*) secara eksekusi langsung (*on-the-fly*).

c). Sistem Manajemen Basis Data

Perangkat lunak basis data relasional yang dioperasikan untuk mengelola persistensi data metadata konten, relasi kategori, dan riwayat interaksi. Sistem ini juga bertugas mengeksekusi kueri struktural (seperti operasi *JOIN* dan kondisional) pada fase penyaringan kandidat awal.

d). *Integrated Development Environment (IDE)*

Lingkungan pengembangan terpadu (seperti Visual Studio Code atau GoLand) yang difungsikan sebagai editor sintaks utama, dilengkapi dengan fitur analisis kode statis, *debugging*, serta integrasi sistem kontrol versi (*version control system*).

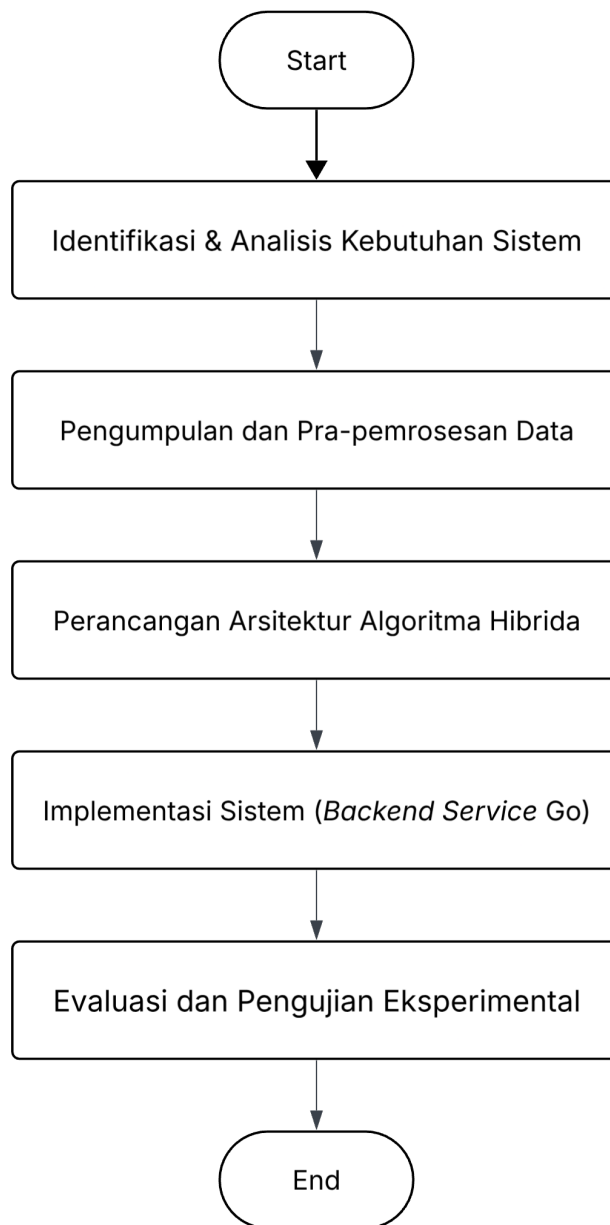
e). Instrumen Pengujian API (*API Testing and Profiling Tools*)

Perangkat lunak klien API (seperti Postman) dan perangkat lunak pengujian beban (*load testing tools*, seperti Apache JMeter) yang digunakan untuk memvalidasi presisi fungsionalitas *endpoint* perankingan, mengukur metrik Waktu Respons (*Response Time*), dan mengevaluasi kapasitas *Throughput* sistem dalam menangani simulasi permintaan serentak (*concurrent requests*).

2.2 Prosedur Penelitian

Penelitian ini mengadopsi metode rekayasa perangkat lunak yang dipadukan dengan pendekatan riset eksperimental komputasional. Pendekatan ini dipilih untuk memastikan bahwa model matematis algoritma perankingan hibrida yang dirancang tidak hanya valid secara teoretis, tetapi juga tangguh dan optimal ketika diimplementasikan ke dalam arsitektur layanan *backend* (API) yang fungsional.

Tahapan sistematis dalam penelitian ini direpresentasikan melalui diagram alir kerangka kerja, yang kemudian dijabarkan ke dalam 5 (lima) langkah operasional. Penjabaran dari masing-masing tahapan pada kerangka kerja di atas adalah sebagai berikut:



Gambar 2.2 Alur kerangka kerja

2.2.1 Tahap Identifikasi dan Analisis Kebutuhan Sistem

Tahap inisiasi ini difokuskan pada analisis fundamental mengenai kelemahan algoritma *feed* konvensional (seperti *chronological* murni atau *trending* murni) yang rentan terhadap bias popularitas dan *filter bubble*. Kegiatan utama pada tahap ini mencakup penentuan parameter sistem, yaitu mengidentifikasi variabel

ketertarikan (*Interest*), variabel metrik keterlibatan (*Engagement Metrics*), dan variabel temporal (*Time-Awareness*). Selain itu, dilakukan juga perumusan kebutuhan arsitektur layanan *Application Programming Interface* (API) yang akan bertugas menyajikan data secara *real-time* ke sisi klien.

2.2.2 Tahap Pengumpulan dan Pra-pemrosesan Data (*Data Preprocessing*)

Tahap ini berfokus pada akuisisi data uji dan transformasi data mentah agar kompatibel dengan lingkungan komputasi algoritma. Proses pra-pemrosesan mencakup dua metrik krusial:

1) Vektorisasi Kategori

Mengonversi data riwayat preferensi pengguna dan entitas kategori konten ke dalam bentuk representasi ruang vektor (*vector space*) sebagai masukan mutlak untuk perhitungan *Cosine Similarity*.

2) Normalisasi Data Interaksi

Menerapkan teknik penskalaan maksimum absolut (*Max-Absolute Scaling*) terhadap nilai agregat interaksi mentah (seperti *Watch Count*, *Used Count*, dan *Like Count*). Proses ini wajib dilakukan untuk menyeragamkan rentang skala kualitas interaksi menjadi rasio 0 hingga 1, sehingga tidak mendominasi variabel lain saat dilakukan pembobotan akhir.

2.2.3 Tahap Perancangan Arsitektur Algoritma

Tahap ini merupakan fase inti yang merancang logika aliran data sistem perankingan, dimulai dari permintaan pengguna hingga penyajian daftar konten yang relevan. Perancangan mengadopsi pola arsitektur rekomendasi dua-tahap (*Two-Stage Recommender System*) [3] yaitu pemisahan proses ke dalam fase

Pembangkitan Kandidat (*Candidate Generation*) dan fase Perankingan (*Ranking*). Pemisahan ini bertujuan menyeimbangkan antara cakupan luas (*broad recall*) pada fase pertama dengan presisi tinggi (*high precision*) pada fase kedua, sebagaimana lazim diterapkan pada sistem rekomendasi berskala besar.

3) Fase Pembangkitan Kandidat (*Candidate Generation*)

Fase pertama berfungsi sebagai corong penyaring awal yang mereduksi ruang pencarian dari skala katalog (jutaan konten potensial) menjadi himpunan kandidat berukuran kecil yang siap dievaluasi pada fase berikutnya. Pengumpulan kandidat dilakukan secara paralel dari tiga kolam sumber untuk mencegah isolasi preferensi pengguna (*filter bubble*) [3]:

a). Kolam Trending (*Trending-Based Pool*)

Kolam ini berisi konten dengan agregasi metrik popularitas tertinggi, yaitu pengurutan menurun berdasarkan kombinasi *rating_avg*, *used_count*, *watch_count*, dan *like_count*. Kolam Trending menjamin keragaman konten populer global hadir di kandidat akhir. Maksimum 300 konten teratas dari kolam ini dimasukkan ke himpunan kandidat awal.

b). Kolam Kebaruan (*Freshness-Based Pool*)

Kolam ini berisi 300 konten paling mutakhir berdasarkan timestamp publikasi, tanpa pertimbangan personalisasi atau popularitas. Kehadiran kolam ini memastikan konten yang baru diunggah memiliki kesempatan masuk ke fase scoring meskipun belum terakumulasi metrik interaksi.

c). Kolam Minat (*Interest-Based Pool*)

Kolam ini menyaring konten berdasarkan kategori-kategori minat tertinggi pengguna berdasarkan profil minat pada tabel *user_category_stats*. Sistem

mengambil 12 kategori dengan skor minat tertinggi, kemudian untuk setiap kategori diambil 100 konten terkait paling mutakhir.

Hasil ketiga kolom digabungkan melalui operasi himpunan-gabungan dengan deduplikasi sehingga setiap konten hanya muncul satu kali pada himpunan kandidat akhir. Total kandidat yang masuk ke fase berikutnya berkisar antara 300 hingga 900 entri tergantung tumpang tindih antar kolom.

4) Fase Perankingan (Hybrid Scoring)

Setiap konten dalam himpunan kandidat dievaluasi melalui formula weighted linear combination dari tiga pilar penilaian, sebagaimana didefinisikan secara teoretis pada Bab III sub-bab 3.2.6:

$$Score_{final}(c) = \omega_1 \cdot Cosine(u, c) + \omega_2 \cdot S_{fresh}(c) + \omega_3 \cdot Quality(c)$$

dengan bobot empiris $\omega_1 = 0,55$, $\omega_2 = 0,25$, dan $\omega_3 = 0,20$. Komponen penilaian dijabarkan sebagai berikut:

- a). . Cosine Similarity ($Cosine(u, c)$) mengukur kemiripan vektor minat pengguna terhadap vektor kategori konten.
- b). *Exponential Time Decay* ($S_{fresh}(c)$) mengukur kebaruan konten dengan parameter peluruhan $\lambda = 72$ jam
- c). Quality Score ($Quality(c)$) kombinasi skor rata-rata penilaian dan engagement metrics ternormalisasi.

5) Fase Pengurutan dan Paginasi

Setelah seluruh kandidat memperoleh nilai $Score_{final}(c)$, daftar diurutkan secara menurun (descending) dengan kunci pengurutan komposit untuk menjamin total ordering:

$ORDER BY Score_{final}(c) DESC \rightarrow t_c DESC \rightarrow ID_{(c)} DESC$

Tie-breaker *created_at* dan *id* mencegah ambiguitas urutan ketika dua atau lebih konten memperoleh skor identik. Setelah pengurutan, sistem menerapkan paginasi berbasis kursor (*cursor-based pagination*) yang memungkinkan pengguna melakukan scroll feed secara berkelanjutan tanpa risiko duplikasi atau lewatan baris. Hasil akhir berupa daftar konten teratas yang dikemas dalam format JSON dan didistribusikan ke aplikasi klien melalui REST API.

6) Penanganan Cold-Start

Untuk pengguna yang belum memiliki riwayat interaksi (cold-start), sistem menerapkan strategi fallback bertingkat yaitu pengguna anonim dilayani melalui trending global, pengguna terautentikasi tanpa riwayat dilayani lewat yang sudah ada, dan pengguna dengan preferensi onboarding mendapat skor dasar pada vektor minat $[N+2]$.

2.2.4 Tahap Implementasi Sistem

Tahap ini merealisasikan rancangan algoritma yang didefinisikan pada sub-bab sebelumnya menjadi sistem operasional yang dapat diakses melalui antarmuka REST API. Implementasi mengikuti paradigma Clean Architecture yang membagi tanggung jawab kode ke dalam lapisan-lapisan terstruktur, dengan tujuan menjaga separation of concerns dan memfasilitasi pengujian unit secara mandiri.

1. Pemilihan Bahasa Pemrograman

Implementasi inti backend menggunakan bahasa pemrograman Go (Golang) versi 1.24. Pemilihan Go didasarkan pada karakteristik teknis yang sesuai dengan kebutuhan sistem perankingan real-time:

- a). Kompilasi statis (statically compiled) yang menghasilkan binary terkompilasi yang dieksekusi langsung tanpa interpreter.
- b). Manajemen konkurensi melalui goroutines memungkinkan pemanggilan paralel ke berbagai kolam kandidat tanpa overhead thread sistem operasi.
- c). Built-in HTTP server yang ringan sehingga memfasilitasi pengembangan REST API tanpa ketergantungan framework eksternal.
- d). Manajemen memori otomatis (garbage-collected) mengurangi risiko *memory leak* tanpa mengorbankan perform

2. Arsitektur Lapisan (Layered Architecture)

Sistem dibagi ke dalam tiga lapisan utama:

a). Lapisan HTTP Handler

Lapisan terluar yang menerima permintaan HTTP dari aplikasi klien (Validasi parameter, decoding cursor pagination, memanggil layanan logika bisnis, serialisasi respons).

b). Lapisan Service

Lapisan tengah yang mengimplementasikan logika bisnis dan dispatching ke algoritma yang sesuai berdasarkan tipe feed yang diminta.

c). Lapisan Repository

Lapisan terdalam yang berinteraksi langsung dengan persistence layer. Seluruh kueri agregasi basis data, perhitungan skor hibrida, dan logika scoring algoritmik dieksekusi pada lapisan ini.

Pembagian tiga lapisan ini sejalan dengan praktik pengembangan sistem berbasis web modern yang berfokus pada keterbacaan dan kemampuan pemeliharaan kode [21].

3. Komponen Pendukung

Sistem operasional bergantung pada dua komponen infrastruktur pendukung, yaitu MySQL sebagai basis data relasional utama yang menyimpan persistensi data dan REST API sebagai antarmuka komunikasi dengan aplikasi klien menggunakan format JSON. Pendekatan REST API berbasis web ini sejalan dengan penelitian Khotimah et al. (2022) [21].

4. Konfigurasi Bobot Algoritma

Untuk memfasilitasi eksperimen sensitivitas dan ablation pada Tahap Evaluasi, bobot ω_1, ω_2 , dan ω_3 pada formula skor hibrida diparametrisasi melalui environment variable. Konfigurasi ini memungkinkan penyetelan ulang bobot tanpa perlu mengompilasi ulang binary, sehingga memudahkan eksperimen iteratif.

2.2.5 Tahap Evaluasi dan Pengujian Eksperimental

Tahap akhir bertujuan memvalidasi secara empiris bahwa sistem yang diimplementasikan benar-benar memberikan kualitas rekomendasi yang lebih baik dibandingkan algoritma baseline konvensional, serta memenuhi tuntutan kinerja sistem yang dipersyaratkan untuk penyajian feed real-time. Validasi dilaksanakan melalui empat skenario pengujian yang saling melengkapi sebagaimana disajikan pada

Tabel 2.2.5 Skenario Pengujian

Kode	Skenario	Tujuan Penelitian	Metrik Utama
S1	Pengujian Akurasi Rekomendasi	Membandingkan kualitas urutan rekomendasi antara sistem hibrida dengan baseline konvensional	NDCG@K, Precision@K, Recall@K
S2	Pengujian Sensitivitas Parameter Bobot	Memvalidasi pemilihan bobot ω_1, ω_2 , dan ω_3 secara empiris	NDCG@10 sebagai fungsi ω
S3	Pengujian Ablation (Kontribusi Komponen)	Memvalidasi kontribusi independen tiap pilar (Cosine, Freshness, Quality)	NDCG@10 per varian konfigurasi
S4	Pengujian Kinerja Sistem	Mengukur latensi dan throughput sistem pada beban kerja bervariasi	Response time (p50, p95, p99), throughput (RPS)

1. Pengujian Akurasi Rekomendasi (Skenario S1)

Skenario ini mengevaluasi kualitas urutan rekomendasi menggunakan tiga metrik standar dari literatur sistem rekomendasi: NDCG@K, Precision@K, dan Recall@K. Pengujian dilakukan terhadap 100 pengguna sintetis yang dibagi

menjadi 5 arketipe, dibandingkan dengan dua baseline: Chronological (feed=catalog) dan Trending Global (feed=trending). (NDCG).

2. Pengujian Sensitivitas Parameter Bobot (Skenario S2)

Skenario ini melakukan *grid search* pada ruang parameter ω_1, ω_2 , dan ω_3 untuk memvalidasi pemilihan bobot yang diusulkan (0,55 / 0,25 / 0,20) Variasi bobot diuji pada interval 0,1 hingga 0,7 dengan kendala ω_1, ω_2 , dan $\omega_3 = 1$ [18].

3. Pengujian Ablation (Skenario S3)

Skenario ini menguji empat varian konfigurasi untuk memvalidasi bahwa setiap pilar memberikan kontribusi independen terhadap performa akhir:

- a). Variant A hanya Cosine (1,00 / 0,00 / 0,00)
- b). Variant B Cosine + Freshness (0,70 / 0,30 / 0,00)
- c). Variant C Cosine + Quality (0,70 / 0,00 / 0,30)
- d). Variant D Hybrid / proposed (0,55 / 0,25 / 0,20)

4. Pengujian Kinerja Sistem (Skenario S4)

Skenario terakhir mengevaluasi kemampuan sistem dalam menyajikan rekomendasi pada beban produksi. Pengujian dilakukan menggunakan tools Apache JMeter 5.6 pada beban berurutan: 1, 10, 50, 100, dan 500 pengguna konkuren, diukur pada kondisi Warm cache dan Cold cache.

5. Lingkungan Pengujian dan Reproduksiabilitas

Seluruh pengujian dieksekusi pada lingkungan tunggal (single-node). Untuk menjamin reproduksiabilitas, seluruh skrip pembangkitan dataset dan prosedur evaluasi menggunakan random seed yang ditetapkan (seed = 42). Skrip eksperimen didokumentasikan dalam bentuk Jupyter Notebook yang dilampirkan pada Lampiran skripsi.

5. DAFTAR PUSTAKA

- [1] M. S. Chelvam, S. Haw, L. D. Krisnawati, dan A. Mahastama, "Hybrid-Based Recommender System Based on Electronic Product Reviews," *JOIV: International Journal on Informatics Visualization*, vol. 9, no. 4, hal. 1752-1764, 2025.
- [2] E. J. de Borba, I. Gasparini, dan D. Lichtnow, "Time-Aware Recommender Systems: A Systematic Mapping," dalam *International Conference on Human-Computer Interaction*, 2021.
- [3] M. Al-Ghuribi, S. A. Noah, dan S. Tiun, "Hybrid Quality-Based Recommender Systems: A Systematic Literature Review," *IEEE Access*, 2024.
- [4] B. S. B. Contreras dan L. A. Digiampietri, "Evolution of Scientific Interests: A TimeAware Recommender System," *Anais do Brazilian Workshop on Social Network Analysis and Mining (BraSNAM)*, 2025.
- [5] A. Y. Hassan, E. Fadel, dan N. Akkari, "Exponential Decay Function-Based TimeAware Recommender System for e-Commerce Applications," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 13, no. 10, 2022.
- [6] M. Rostami *et al.*, "A Novel Time-Aware Food Recommender-System," 2022.
- [7] SAI Org, "Exponential Decay Function-Based Time-Aware Recommender System," 2022.
- [8] Liu *et al.*, "TD-DNN: A Time Decay-Based Deep Neural Network," 2022.
- [9] Pilgram *et al.*, "A consensus privacy metrics," 2025.
- [10] M. Asfi dan N. Fitrianingsih, "Implementasi Algoritma Naive Bayes Classifier sebagai Sistem Rekomendasi Pembimbing Skripsi," *InfoTekJar: Jurnal Nasional Informatika dan Teknologi Jaringan*, vol. 5, no. 1, 2020.
- [11] F. Ricci, L. Rokach, dan B. Shapira, "Recommender systems: introduction and challenges," dalam *Recommender systems handbook*, Boston, MA: Springer, 2015, hal. 1-34.
- [12] F. O. Isinkaye, Y. O. Folajimi, dan B. A. Ojokoh, "Recommendation systems: Principles, methods and evaluation," *Egyptian Informatics Journal*, vol. 16, no. 3, hal. 261-273, 2015.
- [13] E. Çano dan M. Morisio, "Hybrid recommender systems: A systematic literature review," *Intelligent Data Analysis*, vol. 21, no. 6, hal. 1487-1524, 2017.
- [14] J. Han, M. Kamber, dan J. Pei, *Data mining: concepts and techniques*, 3rd ed. Morgan Kaufmann Publishers, 2011.
- [15] P. G. Campos, F. Díez, dan I. Cantador, "Time-aware recommender systems: a comprehensive survey and analysis of existing evaluation protocols," *User Modeling and User-Adapted Interaction*, vol. 24, no. 1-2, hal. 67-119, 2014.

- [16] Y. Ding dan X. Li, "Time weight collaborative filtering," dalam *Proceedings of the 14th ACM international conference on Information and knowledge management (CIKM '05)*, 2005, hal. 485-492.
- [17] K. Järvelin dan J. Kekäläinen, "Cumulated gain-based evaluation of IR techniques," *ACM Transactions on Information Systems (TOIS)*, vol. 20, no. 4, hal. 422-446, 2002.
- [18] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2015.
- [19] A. Jacob, R. T. Subagio, dan A. Nursetyo, "Perangkat Otomatisasi Pergantian Air Budidaya Ikan Lele Menggunakan Metode Rule Based Expert System," *RIGGS: Journal of Artificial Intelligence and Digital Business*, 2025.
- [20] K. A. Pratama, R. T. Subagio, M. Hatta, dan V. Asih, "Implementasi Load Balancing Pada Web Server Menggunakan Apache Dengan Server Mirror Data Secara Real Time," *Jurnal Digit: Digital of Information Technology*, vol. 11, no. 2, hal. 178-189, 2021.
- [21] U. Khotimah, J. Z. Mutaqin, P. Sokibi, R. Adam, V. Asih, S. Santoso, dan W. Ilham, "Perancangan Sistem Informasi Pelayanan Administrasi Kependudukan Desa Candrajaya Berbasis Web," *Jurnal Pengabdian UCIC*, vol. 1, no. 2, 2022.
- [22] S. G. Andika, K. Kusnadi, dan P. Sokibi, "Sistem Pendukung Keputusan Pemilihan Kegiatan Ekstrakurikuler Untuk Siswa Sma Menggunakan Metode Simple Multi Attribute Rating Technique (Studi Kasus: Sma Santa Maria Cirebon)," *Jurnal Digit: Digital of Information Technology*, vol. 9, no. 1, hal. 59-70, 2020.